

ASSIGNMENT

Recursion and Backtracking:

Problem1 : Understanding Recursion and mechanics

(a) Recursive Function Structure

To calculate the total size of a directory, recursion can be used because directories may contain other directories.

Base Case

If the current item is a file, return its size.

Recursive Case

If the current item is a directory, recursively calculate the size of all files and subdirectories inside it and add them together.

Pseudocode

```
function getDirectorySize(directory):
```

```
totalSize = 0
```

```
for each item in directory:
```

```
    if item is a file:
```

```
        totalSize = totalSize + item.size
```

```
    else if item is a directory:
```

```
        totalSize = totalSize + getDirectorySize(item)
```

```
return totalSize
```

(b) Trace of Execution (Call Stack)

Directory Structure:

project/

|

|— src/

| |— main.java (100 KB)

| |— utils.java (50 KB)

|

|— docs/

| |— readme.txt (10 KB)

| |— guides/

| |— setup.pdf (200 KB)

|

└─ config.xml (20 KB)

Execution Steps

getDirectorySize(project) is called.

It enters src/

main.java = 100 KB

utils.java = 50 KB

Total for src = 150 KB

It enters docs/

readme.txt = 10 KB

enters guides/

setup.pdf = 200 KB

Total for docs = 210 KB

config.xml = 20 KB

Final Calculation

Total Size = $150 + 210 + 20$

Total Size = 380 KB

(c) Time Complexity

Let n = total number of files and directories.

Each file and directory is visited exactly once.

Therefore,

Time Complexity = $O(n)$

Space Complexity

The recursion uses a call stack based on the depth of the directory tree.

Space Complexity = $O(h)$

where h = height of the directory tree.

(d) Iterative Solution

Yes, this problem can also be solved using an iterative approach instead of recursion.

A stack can be used to simulate recursion.

Iterative Pseudocode

```
function getDirectorySizeIterative(root):
```

```
    stack.push(root)
```

```
    totalSize = 0
```

```
    while stack is not empty:
```

```
        current = stack.pop()
```

```
        for each item in current:
```

```
            if item is a file:
```

```
                totalSize = totalSize + item.size
```

```
            else if item is a directory:
```

```
                stack.push(item)
```

```
    return totalSize
```

Comparison

Approach	Method	Risk
Recursive	Uses function calls	Stack overflow for deep directories
Iterative	Uses explicit stack	More control over memory

(e) Symbolic Link Cycle Problem

If a symbolic link creates a cycle, the recursion may enter an infinite loop.

Example:

Directory A → Directory B → Directory C → Directory A

This creates infinite recursion.

Solution

To prevent this, maintain a set of visited directories.

Modified Pseudocode

```
function getDirectorySize(directory, visited):
```

```
    if directory in visited:
```

```
        return 0
```

```
    visited.add(directory)
```

```
    totalSize = 0
```

```
    for each item in directory:
```

```
        if item is a file:
```

```
            totalSize = totalSize + item.size
```

else if item is a directory:

totalSize = totalSize + getDirectorySize(item, visited)

return totalSize

The visited set ensures that each directory is processed only once, preventing infinite recursion.

Problem 2: N-Queens Optimization and Variants

Scenario

A chess tutorial application needs to demonstrate the **N-Queens problem**, where N queens must be placed on an N×N chessboard such that **no two queens attack each other**.

Rules:

- No two queens share the same **row**
- No two queens share the same **column**
- No two queens share the same **diagonal**

For a standard **8×8 chessboard**, this becomes the **8-Queens Problem**.

a) Search Space Analysis for 8-Queens

1. Brute Force Search Space

Without any constraints, each square could either contain a queen or not.

Total possible placements:

$$2^{(N^2)}$$

For $N = 8$:

$$2^{(8^2)} = 2^{64}$$

= 18,446,744,073,709,551,616 possible board states

This is extremely large and impractical to check.

2. With “One Queen per Row” Constraint

Instead of placing queens anywhere, we place **exactly one queen per row**.

Each row has **8 possible columns**.

Total possibilities:

$$8^8$$

= 16,777,216 combinations

This drastically reduces the search space.

3. Estimated Positions Explored with Backtracking

Backtracking eliminates positions that violate constraints early.

Actual explored states are much smaller.

Approximate number explored:

≈ 15,000 – 20,000 states

This pruning makes the algorithm efficient enough for $N = 8$.

b) Optimized N-Queens Using 1D Array

Instead of a 2D board, we store queen positions in a **1D array**.

Example:

```
board[row] = column
```

Example placement:

```
board = [1, 3, 0, 2]
```

Meaning:

Row 0 → column 1 Row 1 → column 3 Row 2 → column 0 Row 3 → column 2

Additional Tracking Arrays

To quickly detect attacks:

```
column[n]  
diag1[2n]  
diag2[2n]
```

Where:

- `column[c] → column occupied`
- `diag1[row + col]`
- `diag2[row - col + n]`

Optimized Pseudocode

```
function solve(row):

    if row == N:
        print solution
        return true

    for col = 0 to N-1:

        if column[col] == false AND
           diag1[row+col] == false AND
           diag2[row-col+N] == false:

            board[row] = col

            column[col] = true
            diag1[row+col] = true
            diag2[row-col+N] = true

            solve(row + 1)

            column[col] = false
            diag1[row+col] = false
            diag2[row-col+N] = false
```

c) Trace for N = 4

Goal: place **4 queens**.

One valid solution:

Row 0 → Column 1

Row 1 → Column 3

Row 2 → Column 0

Row 3 → Column 2

Board representation:

. Q . .

. . . Q

Q . . .

. . Q .

Step-by-Step Execution

Step 1

Place queen:

row 0 → column 1

Step 2

Try row 1:

column 0 → conflict column 1 → conflict column 2 → diagonal conflict column 3 → valid

Place queen:

row 1 → column 3

Step 3

Row 2:

column 0 → valid

Place queen:

row 2 → column 0

Step 4

Row 3:

column 2 → valid

Place queen:

row 3 → column 2

Solution found.

Backtracking Points

Backtracking occurs when:

- A row has **no valid column**
- The algorithm returns to the previous row
- It tries the **next column**

d) Finding Only ONE Solution Faster

Normally N-Queens searches **all solutions**.

To stop after the first solution:

Modify recursion:

```
if solve(row + 1):  
    return true
```

This stops the search once a valid board is found.

This significantly **reduces execution time**.

e) N-Queens with Forbidden Squares

New rule:

Some squares are **blocked** and cannot contain queens.

Example:

X = forbidden square

Algorithm modification:

Before placing a queen:

```
if board[row][col] is forbidden:  
    skip
```

Updated condition:

```
if column[col] == false AND  
   diag1[row+col] == false AND  
   diag2[row-col+N] == false AND  
   not forbidden[row][col]
```

Impact on Complexity

Worst-case complexity remains:

$O(N!)$

However:

Forbidden squares reduce valid positions, which **prunes the search tree earlier** and may improve practical performance.

Final Conclusion

The N-Queens problem demonstrates the power of **backtracking optimization**.

Key improvements include:

- Reducing search space using constraints
- Using **1D arrays instead of 2D boards**
- Tracking columns and diagonals for constant-time checks
- Early termination when only one solution is required
- Handling additional constraints like forbidden squares

These optimizations make the algorithm efficient even for larger board sizes.

Problem 3: Backtracking Template Application

Scenario

A mobile gaming company is developing a word puzzle game. Players must form words by connecting adjacent letters in a **4×4 grid**. A letter can be used **only once** in a word. Movement is allowed **horizontally, vertically, and diagonally**.

Given Grid

C	A	T	S
O	R	E	A
D	E	A	M
E	L	L	S

Target Word: "**DREAM**"

a) Backtracking Algorithm (Pseudocode)

Backtracking explores possible paths step by step and **undoes choices** when a path cannot form the target word.

State Representation

- Grid position (row, col)
- Current index in the word
- Visited cells (to avoid reusing letters)

Constraints

- Movement allowed in **8 directions**
- Each cell can be used **only once**

Goal Condition

- When all characters of the word "**DREAM**" are matched.

Pseudocode

```
function searchWord(board, word):  
    rows = number of rows  
    cols = number of columns
```

```
create visited[rows][cols]

for i = 0 to rows-1:
    for j = 0 to cols-1:
        if board[i][j] == word[0]:
            if backtrack(i, j, 0):
                return true
```

```
return false
```

```
function backtrack(row, col, index):
```

```
    if index == length(word):
        return true
```

```
    if row < 0 OR col < 0 OR row >= rows OR col >= cols:
        return false
```

```
    if visited[row][col] == true:
        return false
```

```
    if board[row][col] != word[index]:
        return false
```

```
    visited[row][col] = true
```

```
    for each direction in 8 directions:
        if backtrack(newRow, newCol, index + 1):
            return true
```

```
    visited[row][col] = false
```

```
return false
```


b) Decision Tree for Finding "DREAM"

We start searching for **D** in the grid.

Possible starting point:

D at (2,0)

Path exploration:

D → R → E → A → M

Successful Path

(2,0) D

(1,1) R

(1,2) E

(2,2) A

(2,3) M

Visual path:

C A T S

O R* E* A

D* E A* M*

E L L S

* indicates letters used in the successful path.

c) Execution Trace

Step-by-step exploration:

1. Start at **D (2,0)**
2. Search neighbors → find **R (1,1)**
3. Next letter **E** → found at **(1,2)**
4. Next letter **A** → found at **(2,2)**
5. Next letter **M** → found at **(2,3)**

Word formed successfully.

Backtracking Case

If a chosen neighbor does not match the next character:

- The algorithm **returns false**
- It **undoes the last choice**
- Then tries another neighbor

This process continues until a valid path is found or all possibilities fail.

d) Worst-Case Time Complexity

Let:

$N \times M$ = grid size L = word length

From each cell we can move in **8 directions**.

Worst-case branching factor:

8

Total recursive calls:

$O((N \times M) \times 8^L)$

Explanation:

- We try every grid cell as a starting point

- From each step we explore up to **8 neighbors**

For a **4×4 grid** and word length **5**, worst case is manageable.

e) Finding ALL Possible Paths

Currently the algorithm **stops after the first successful match**.

To find **all possible ways** to form the word:

Required Changes

1. Remove the early return `true`.
2. Store successful paths in a list.
3. Continue exploring after finding a match.

Modified Logic

```
if index == length(word):  
    store current path  
    return
```

Instead of stopping, the algorithm continues exploring other paths.

Additional Data Structure Needed

```
List<List<Cell>> allPaths
```

This stores every valid sequence of grid coordinates that forms the word.

Final Conclusion

Backtracking efficiently solves the word search problem by:

- Exploring possible paths
- Checking constraints
- Undoing incorrect choices

The algorithm guarantees correctness while reducing unnecessary exploration compared to brute force. It is well suited for puzzle games where all possible combinations must be explored systematically.